

Application of reinforcement learning based on curriculum learning for the pipe auto-routing of ships

Yongsu Kim, Kyungho Lee*, Byeongwook Nam and Youngsoo Han 

Department of Naval Architecture & Ocean Engineering, Inha University, Michuhol-gu, Incheon 22212, Republic of Korea

*Corresponding author. E-mail: kyungho@inha.ac.kr

Abstract

The pipe routing of ships has been manually performed by experts, and the design quality depends on the competence of the experts. Therefore, studies on pipe-routing automation and optimization are required. In addition, the pipe-routing task in a ship that requires frequent pipe-routing modifications requires a long time to be optimized. In this study, we developed a methodology that enables a rapid response in situations where frequent pipe-routing modifications are required by applying curriculum learning that can be stably learned by gradually solving easy-to-complex problems. In addition, this study aimed to minimize the length of the pipe and number of bends as an objective function. Finally, the proposed methodology was verified by comparing it with existing studies that used the A*, jump point search, and reinforcement-learning algorithms to determine the search speed, number of bends, and length of the path.

Keywords: pipe auto-routing, path finding algorithm, reinforcement learning, curriculum learning

1. Introduction

Piping in the shipbuilding and offshore industries is designed using various electrical devices and instruments (Lee et al., 2019). Therefore, many factors need to be considered in piping routing, a process that occupies up to 50% of the total design time (Park & Storch, 2002). Moreover, as shown in Fig. 1, the number of piping types has increased, and the regulations for pipe routing have been strengthened, further complicating piping design work (Shin et al., 2020).

In addition, current pipe-routing work complies with constraints and regulations, and several functions are performed manually according to expert knowledge. Thus, pipe routing is time intensive and its quality depends on the competence of the expert. Therefore, an automation and optimization method that can increase the efficiency of complicated and burdensome pipe routing is required. To solve this problem, several studies have been conducted on the automation of pipe routing by using path-search algorithms, optimization algorithms, and learning. However, if equipment is added and the location is modified, according to previous studies, a large amount of time is consumed because the route must be searched based on the modifications (environmental changes). Therefore, we propose a curriculum-learning-based reinforcement-learning method that can respond to environmental changes by dynamically discovering routes during pipe routing and creating relatively easy and complex environments.

2. Related Research

Recently, various algorithms have been used to reflect not only the length of the pipe, but also the number of bends and space avail-

ability in pipe routing. Depending on the route search method, these algorithms fall in two categories: graph-based algorithms and algorithms based on optimization techniques.

Studies have been conducted using a path-search algorithm. Kim (2014) considered a method that allows pipes to pass through a preferred space by setting existing supports or walls as the preferred spaces and studied it using the A* algorithm. Lee et al. (2019) used a method to secure space availability by adding weights according to equipment and proximity using Dijkstra's algorithm. Min et al. (2020) considered a method for solving the search speed problem when the number of nodes increased and studied it by using the jump point search (JPS) algorithm.

Studies have also been conducted using optimization algorithms. Furuholm et al. (2010) and Sui and Niu (2016) defined a pipe-routing problem by minimizing the length of the pipe and the number of bends and solved it using a genetic algorithm (GA). GA derives an optimal solution by generating random solutions and passing them to the next generation by crossover and mutation. Kimura and Ikehira (2009) and Kimura (2011) considered the accessibility of the valve and studied it using a GA. Fan et al. (2006) and Jiang et al. (2015) planned to secure space availability for pipe maintenance and installation, and studied it using the ant colony optimization (ACO) algorithm. The ACO algorithm helps determine the best solution by increasing the probability of searching for an effective solution in the next step by leaving a pheromone in the best solution at every step, and this process is repeated. Shin et al. (2020) considered applicable methods for environmental changes (existing equipment location and direction were modified) and studied them using reinforcement learning.

As aforementioned, prior studies have defined and optimized the objective functions and constraints for pipe auto-routing.

Received: May 9, 2022. Revised: December 13, 2022. Accepted: December 13, 2022

© The Author(s) 2023. Published by Oxford University Press on behalf of the Society for Computational Design and Engineering. This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (<https://creativecommons.org/licenses/by-nc/4.0/>), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited. For commercial re-use, please contact journals.permissions@oup.com



Figure 1: Pipeline in ship.



Figure 2: Definition of the 3D node.

However, in the case of existing studies related to pipe routing, when path modification is required, the path is optimized by calculating the heuristic value of the neighboring node to search the path or go through an optimization process once per path. This process takes considerable time to perform. To solve these problems, a curriculum that can improve performance and solve complicated problems by gradually learning the model through easy-to-difficult steps, as proposed by Narvekar *et al.* (2020) and Narvekar (2017), was used in this study. Therefore, we present a pipe-routing method that applies learning-based reinforcement learning.

3. Assumptions and Objective Functions for Pipe Routing

3.1. Node-based environment configuration and constraint definition

Pipe routing is performed in a node-based environment. A node-based environment implies the division of a space to be routed into a node structure (Fig. 2), and defines each node point as a point through which the pipe passes. The node comprises $Node_{x,y,z} = \{x, y, z\}$. $\{x, y, z\}$ denote x -, y -, and z -coordinates, respectively. The pipe size is configured to fill a single node. In addition, the pipe could be bent at any angle, but only the angle of 90° was considered, and the thickness was not considered.

Unity, a game engine that exhibits two-dimensional (2D)- and 3D-game-development environments, was used to quickly implement the environment and pipe visualization, and the ML-Agent library provided by Unity was used to develop a reinforcement-learning model. The ML-Agent library facilitates the creation of a reinforcement-learning environment in Unity and creates reinforcement-learning models using built-in or external algorithms. The node = {100 100 100} environment was configured using the Unity engine. It includes 1000 000 nodes ranging from $Node_{0,0,0}$ to $Node_{99,99,99}$. In addition, the equipment and piping were considered in a box shape for ease of implementation. The computing power specifications are listed in Table 1.

3.2. Objective function definition in a node-based environment

In this study, the existing equipment was defined as an obstacle node and routing was performed (Fig. 3a). In addition, when one

Table 1: Computing power.

	Value
CPU	Inter® Core i9-9900k CPU @ 3.60 GHz
GPU	NVIDIA GeForce GTX 2080 8.0 GB
RAM	64 GB

pipe path was created, the generated pipe was set as an obstacle node (Fig. 3b).

The objective function is defined in Equation (1) based on the minimization of the total pipe length and number of bends. In Equation (1), the “path length” indicates the total pipe length and the “bend amount” indicates the number of bends in the pipe.

$$\text{Minimize (Path length + Number of bends)} \quad (1)$$

4. Reinforcement Learning and Curriculum Learning

4.1. Reinforcement learning

Reinforcement learning, illustrated in Fig. 4, is a machine-learning technique that focuses on training an algorithm following the cut-and-try approach. The algorithm (agent) evaluates the current situation (state), performs an action, and receives feedback (reward) from the environment after each action. Positive feedback is a reward (in its usual meaning for us) and negative feedback is a punishment for making a mistake. It exhibits high learning performance for more complex problems and continuous state-action space problems (Chun *et al.*, 2022; Oh *et al.*, 2022).

In reinforcement learning, the probability of an action selected according to a state is called a policy and is expressed as Equation (2):

$$\pi(s, a) = P[A_t = a | S_t = s]. \quad (2)$$

In Equation (2), the right-hand side represents the probability that an agent in state s will take action a at time t . In reinforcement learning using neural networks, the policy is determined by the neural network if it is true for policy-based reinforcement learning. Therefore, the policy can be expressed as $\pi(a|s)$.

Reinforcement learning is primarily achieved using the deep Q-network (DQN) algorithm (Mnih *et al.*, 2013), which applies a neural network to approximate the Q functions for each action and state, thereby saving considerable computational resources. Another algorithm, the actor-critic algorithm (Konda & Tsitsiklis, 2003), considers policy- and value-based methods by having separate network approximations for values and behaviors to normalize them and achieve stable results. The trust region policy optimization algorithm (Schulman *et al.*, 2017), proximal policy optimization (PPO; Schulman *et al.*, 2017) using the KL divergence technique.

However, because the DQN uses replay memory, it requires a large amount of memory and has limitations regarding the use of old data for learning. The actor-critic algorithm is on-policy, and data efficiency is low because the data used are discarded immediately. This study has several limitations in terms of implementation. Therefore, for complex pipe routing, the PPO algorithm, which is relatively easy to implement and can quickly converge to the optimal result owing to the reduced number of calculations, was used.

Here, the PPO employs a clipped surrogate objective to prevent the new policy from straying away from the old policy. The clipped

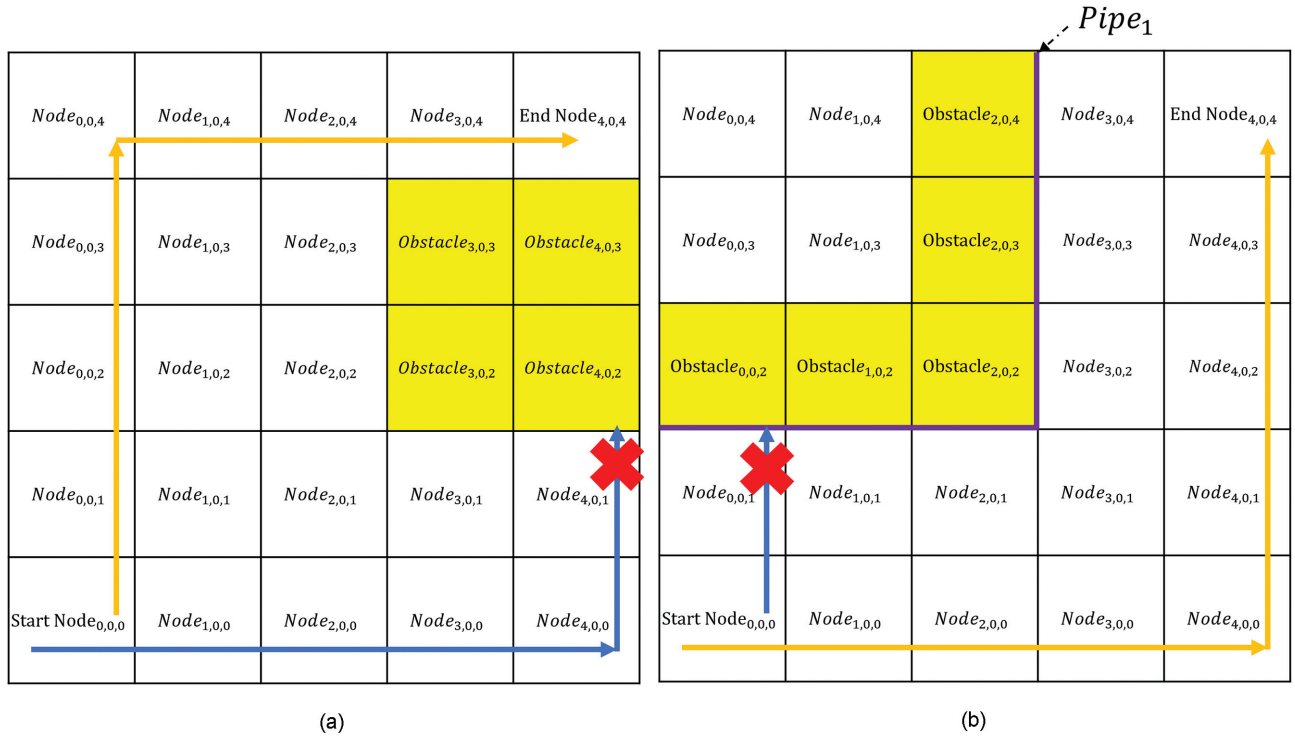


Figure 3: (a) Avoiding the obstacle node and (b) avoiding the installed pipe node.

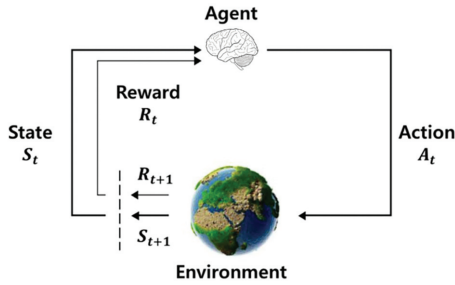


Figure 4: Reinforcement learning.

objective function of state-action (S_t, a_t) is

$$L^{\text{CLIP}}(\theta) = \min(r_t(\theta, \theta_{\text{old}}) A_t, \text{clip}(r_t(\theta, \theta_{\text{old}}), \epsilon) A_t). \quad (3)$$

In Equation (3), θ and θ_{old} are the parameters of the new and old policies, respectively; $r_t(\theta, \theta_{\text{old}})$, where $r_t(\theta, \theta_{\text{old}}) = \frac{\pi_{\theta}(a_t|S_t)}{\pi_{\theta_{\text{old}}}(a_t|S_t)}$ is the probability ratio. We omit explicitly writing the parameter of the old policy θ_{old} . $S_t \sim p^{\pi_{\theta_{\text{old}}}}$, and $a_t \sim \pi_{\theta_{\text{old}}}$ are the sampled states and actions, respectively. A_t is the estimated advantage value of $A \pi_{\theta_{\text{old}}}(S_t, a_t)$. The clipping-function clip is defined as

$$\text{clip}(r_t(\theta, \theta_{\text{old}})) = \begin{cases} 1 - \epsilon, & r_t(\theta) \leq 1 - \epsilon. \\ 1 + \epsilon, & r_t(\theta) \geq 1 + \epsilon. \\ r_t(\theta) & \text{else.} \end{cases} \quad (4)$$

In Equation (4), $(1 - \epsilon, 1 + \epsilon)$ is called the clipping range, and $0 < \epsilon < 1$ is the parameter. The overall objective function is expressed as follows:

$$L^{\text{CLIP}}(\theta) = \frac{1}{T} \sum_{t=1}^T L_t^{\text{CLIP}}(\theta). \quad (5)$$

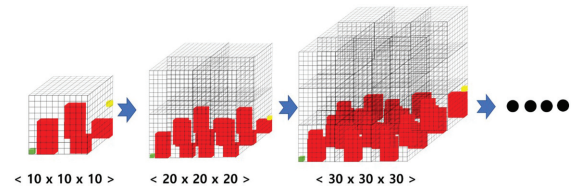


Figure 5: Curriculum learning.

Thus, by reducing the range of update sizes using clips, the amount of computation required to update the policy can be reduced. Simultaneously, reducing the range of update sizes has a reliable learning effect as it effectively limits the update range.

4.2. Curriculum learning

Curriculum learning is illustrated in Fig. 5, where the learning environment is divided into several stages and is gradually trained by increasing the number of stages (Bengio et al., 2009). Curriculum learning is expressed by Equation (6) as follows:

$$Q_{\lambda} \propto W_{\lambda}(z) P(z) \forall z. \quad (6)$$

In Equation (6), $P(z)$ denotes the distribution of data where the user wants to predict the pattern, and $W_{\lambda}(z)$ expresses the progress of the curriculum in the form of a ratio. When $W_{\lambda}(z)$ reaches 1, the entire curriculum is complete. Accordingly, when the entire curriculum is complete, the target that the user wants to optimize becomes similar to the actual data distribution, and stable learning becomes possible. For example, in Fig. 5, if a path search is performed in the $< 30 \times 30 \times 30 >$ environment or a larger environment (more than 27 000 nodes), it is often impossible to reach the highest value endpoint, and information about all nodes and whether to receive the greatest reward may not be adequately learned.

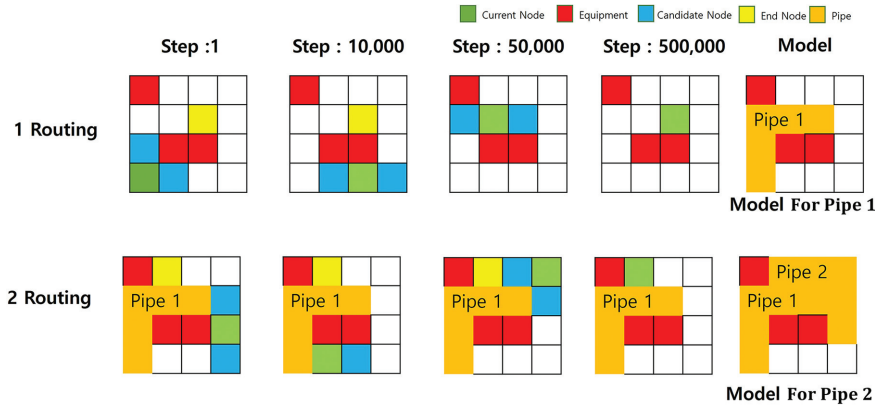


Figure 6: Existing reinforcement learning example.

Therefore, to solve this problem, in Fig. 5, if a path search is performed in 1000 nodes of the $< 10 \times 10 \times 10 >$ environment to acquire node and reward information at an early stage to obtain the information above a certain level, the next environment $< 20, 20, 20 >$ continues the learning through the model trained in the previous environment. Thus, by increasing the number of nodes by 10 on each axis, stable learning is achievable by updating the node and reward information.

5. Pipe Routing Using Reinforcement Learning Based on Curriculum Learning

If a new obstacle is added or the locations of the starting and target points are changed, the route must be re-searched and re-learned. It takes a long time to optimize pipe routing. Therefore, an optimal piping route search was performed by establishing a curriculum-learning-based reinforcement-learning model to respond to such environmental changes.

5.1. Sensing

In reinforcement learning, state selection determines the quality of the result because the agent accepts the state as a learning factor. Shin *et al.* (2020) described a method in which an agent learns the state using the positional relationship between the current and the arrival nodes. However, in this case, it becomes difficult to respond to environmental changes because the model cannot be recycled, and a new model must be developed for each path. A visualization of the existing method is depicted in Fig. 6. Suppose that we select two paths, as shown in Fig. 6. In this case, we build a model that selects the first path in a specified environment. Subsequently, when selecting the second pipe, the first path is added to the environment, and the second path is selected in the environment. Whenever there is a change in the environment (such as adding a pipe), it is necessary to train a new model.

Therefore, a sensor has been introduced to overcome this limitation. As shown in Fig. 7, the object (shooting the ray) attached to the sensor shoots a ray in a specified direction to determine what object (catch some the ray) is hit by the ray and returns the distance from the ray. In this study, as shown in Fig. 8, 10 sensors were attached to the agent at angles of 45° , up and down. The objects attached to the sensors emitted rays, and the distances of the object with which the rays collided were obtained. The object hit by the ray was selected as an obstacle, target point, or wall, and this was expressed as an index for distinguishing the hit ob-

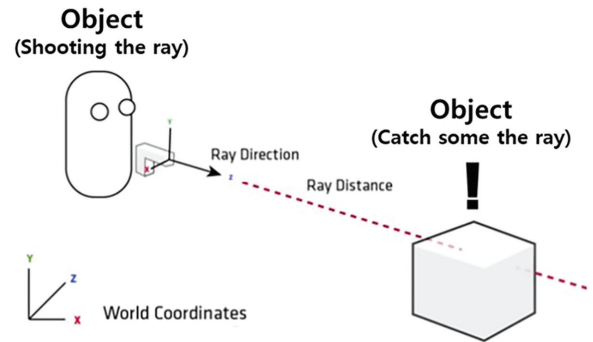


Figure 7: Concept of a ray.

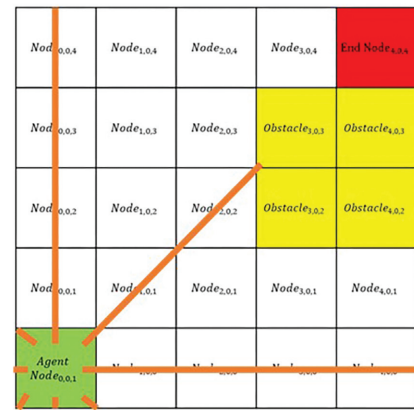


Figure 8: Sensing the object.

ject. The index of the hit object was set as follows: obstacle = 0, wall = 1, and end node = 2.

5.2. Reduced elbow

The bend is changed whenever the path direction is changed, and the objective function is obtained by providing a negative compensation when a bent shape appears, as shown in Fig. 9. Here, a negative reward is an indicator of bad behavior when searching for a path. This can be expressed as Equation (7), which indicates that the reward is allotted according to the action in reinforcement learning, the pre-vector is the value obtained from the difference between the previous and current nodes, and the

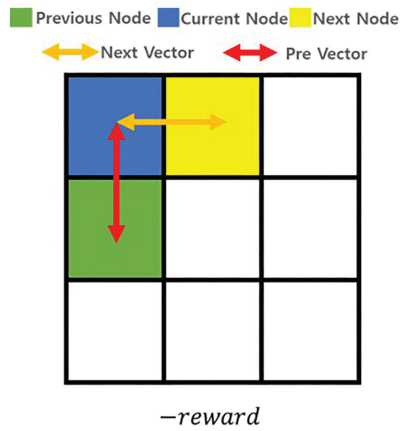


Figure 9: Avoiding the elbow shape.

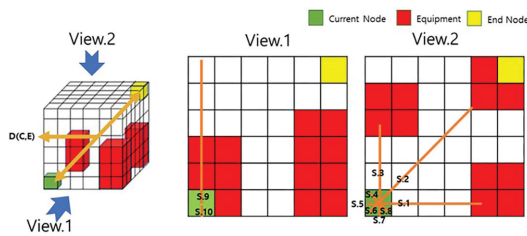


Figure 10: Sensing the environment.

Table 2: Observation parameter.

Input	S.1	S.2	S.3	S.4	S.5	S.6	S.7	S.8	S.9	S.10	D(C, E)
Distance	4	5.6	3	0	0	0	0	0	0	0	8.48
Index	0	0	0	1	1	1	1	1	1	1	X

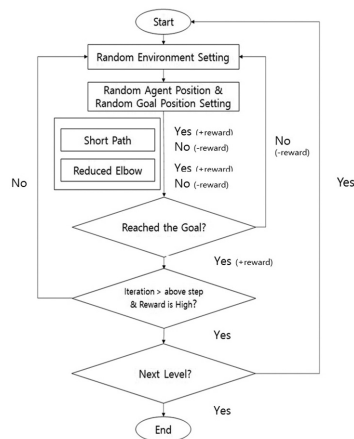


Figure 11: Reinforcement learning with a curriculum-learning algorithm diagram.

next vector is the value obtained from the difference between the current and next nodes:

$$\text{If Dot (Pre Vector, Next Vector)} = 0, \quad -\text{reward}. \quad (7)$$

5.3. Learning

In Sections 5.1 and 5.2, techniques for learning and techniques and concepts for calculating the optimal objective function are described. This section describes the learning of input data and processes.

5.3.1. Input data for learning

As aforementioned, the size of the environment is gradually increased in curriculum learning. Therefore, the size of the environment cannot be used as the input data. Hence, the input data were the positional relationships obtained using the sensors described in Section 5.1. In addition, if a sensor does not identify the end-point, the agent's current location and distance to the endpoint are provided as learning inputs because it does not know whether the agent is going to the endpoint.

The input data are shown in Fig. 10. Figure 10 is divided into View.1 (front view) and View.2 (top view) so that the sensing data can be easily understood. In the figure, S.9–10 refer to the sensor data shot up and down when viewed from the front, and S.1–8 refer to the sensor data shot at an angle of 45° from the top. Here, the sensor receives the distance and the type of object hit as a number. D(C, E) is the distance between the current and end nodes. The data in Fig. 10 are summarized in Table 2.

5.3.2. Learning process

In this study, when the system was started, the size and position of obstacles and the agent and target points were randomly set to create various environments. Subsequently, the agent received positional relationship information for the obstacles, walls, and end nodes as a state through the sensor. Based on the received state, it performed one of the following six actions: it could move up, down, right, left, forward, or backward. The agent was rewarded for its action: it received a reward of 0.2 if it moved in a straight line and 0.1 if it approached the end node. However, if a bent shape appeared, a reward of -0.2 was received, and if it moved away from the end node, the process of receiving a reward of -0.1 was repeated. If the agent did not arrive at the end node during the max step (which is a predetermined constant parameter so that computation time is not wasted), it received a reward of -10 and returned to the environmental-configuration position. However, if it reached the target in a step lower than the maximum step, it received a reward of $+10$. If the total iteration was lower than the maximum iteration or the average reward score was lower than the average reward of the last 10 episodes, it returned to the configuration position. If it exceeded the average reward, the agent entered the next level or the process was terminated. The learning process is illustrated in Fig. 11.

5.3.3. Learning-process visualization

The preceding section elucidates the learning process by using illustrations. Step 1 in Fig. 12 shows a process that moves from the starting point to the endpoint in a random manner. In reinforcement learning, a path is searched by randomly selecting a behavior for a certain period, and the behavior is learned using previous experience as a certain step period passes. Therefore, it is difficult to reach the endpoint. However, if we learn approximately 10 000 steps, we can gradually approach the endpoint. In addition, if we learn approximately 50 000 steps, we can optimally reach the endpoint faster without getting lost. Subsequently, if the accumulated value exceeds a certain level, the model proceeds to the next level (environment size increases) and continues to learn. We aimed

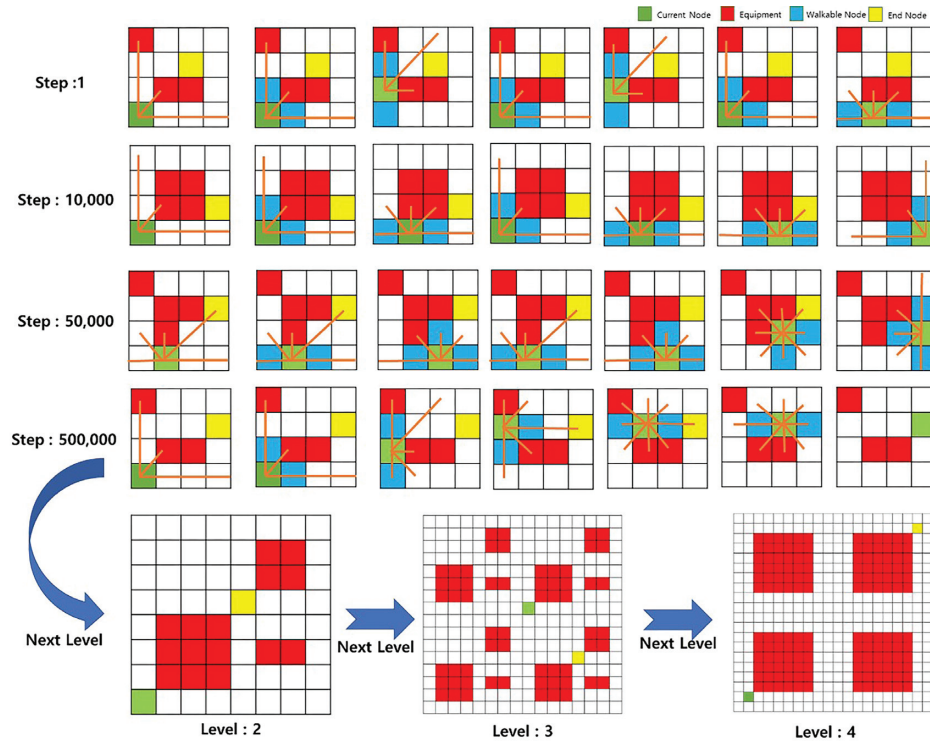


Figure 12: Reinforcement learning with a curriculum-learning algorithm.

Table 3: Hyperparameter.

Parameter	Hyper. 1	Hyper. 2	Hyper. 3	Hyper. 4	Hyper. 5	Hyper. 6	Hyper. 7	Hyper. 8	Hyper. 9
Trainer	PPO	PPO	PPO	PPO	PPO	PPO	PPO	PPO	PPO
Batch size	1024	1024	256	512	1024	1024	1024	1024	1024
Beta	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$	$5.0e-3$
Buffer size	10 240	10 240	2560	5120	10 240	10 240	10 240	10 240	10 240
Epsilon	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.2	0.1
Gamma	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.9	0.95
Hidden units	512	512	512	512	512	256	512	1024	512
Number of hidden layer	3	3	3	3	3	3	3	3	3
Learning rate	0.005	0.0005	0.000 05	0.000 05	0.000 05	0.000 05	0.000 05	0.000 05	0.000 05
Lambda	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95

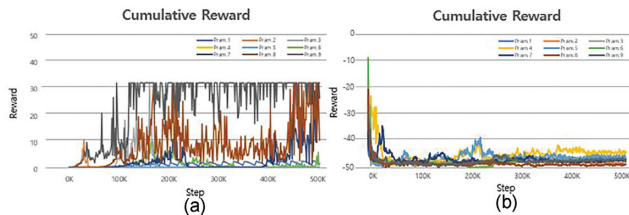


Figure 13: Cumulative reward per hyperparameter. (a) Curriculum and (b) $100 \times 100 \times 100$.

to determine the optimal path for all environments by gradually adapting to environmental changes.

5.3.4. Selection of learning parameters

In this study, for parameter selection, the parameter that yielded the most significant reward was used by comparing the reward functions obtained in 500 000 learning processes. The learning pa-

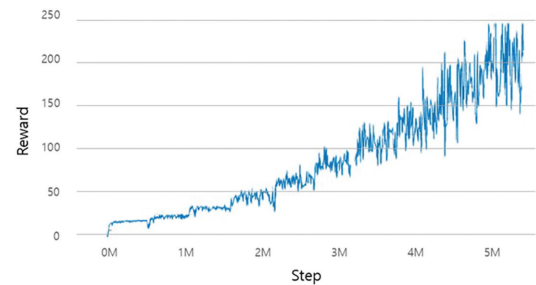


Figure 14: Cumulative reward by reinforcement learning with a curriculum-learning algorithm.

rameters for the reward function are listed in Table 3, and the reward function is presented graphically in Fig. 13a. Parameters such as the learning rate, batch size, gamma, and epsilon were adjusted, and parameter 9, which obtained the best reward value

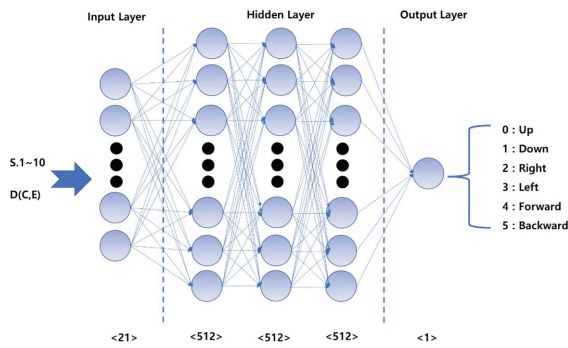


Figure 15: Neural network structure.

with minimum time, was set as the learning parameter. Fig. 13b shows the results of curriculum learning from the beginning in a $100 \times 100 \times 100$ environment. Curriculum learning starts from a random environment arrangement (e.g., change in size and nozzle position of equipment). As this was not performed appropriately, the accumulated rewards gradually decrease.

Curriculum learning was conducted using parameter 9, which yielded the best results for learning, and the corresponding results are shown in Fig. 14. Fig. 14 shows the cumulative compensation value per step from the $< 10 \times 10 \times 10 >$ environment to the $< 100 \times 100 \times 100 >$ environment. As shown, the cumulative reward increases as learning progresses, but does not converge to a certain value. As pipe routing is performed in a randomly generated environment, if the length of the start and end nodes is short, the compensation is low, and if the length is long, the com-

Table 4: Environmental conditions for performance analysis of the proposed method.

Environment	Item.	Point	X-size	Y-size	Z-size	Item.	Point	X-size	Y-size	Z-size
A	Start nozzle.1	X: 13.14, Y: 10.28, Z: -4.50				Start nozzle.2	X: 13.14, Y: 10.28, Z: -5.75			
	End nozzle.1	X: 19.01, Y: 11.94, Z: -7.09				End nozzle.2	X: 19.01, Y: 11.94, Z: -9.46			
	Equipment.1	X: 11.95 Y: 12.10, Z: -8.93	2.42	3.98	3.07	Equipment.2	X: 19.47, Y: 11.12, Z: -7.45	3.61	2.06	1.37
	Equipment.3	X: 19.47, Y: 11.12, Z: -10.11	3.61	2.06	1.37	Equipment.4	X: 12.28, Y: 11.01, Z: -4.85	0.64	1.80	0.70
	Equipment.5	X: 12.28, Y: 11.01, Z: -6.00	0.64	1.80	0.70					
B	Start nozzle.3	X: 20.88, Y: 10.52, Z: -3.62				Start nozzle.4	X: 20.88, Y: 10.52, Z: -4.91			
	End nozzle.3	X: 11.66, Y: 12.55, Z: -12.40				End nozzle.4	X: 12.28, Y: 11.94, Z: -15.87			
	Equipment.6	X: 12.22 Y: 11.90, Z: -12.45	1.80	1.25	1.00	Equipment.7	X: 11.19 Y: 11.12, Z: -15.08	3.61	2.06	1.37
	Equipment.8	X: 22.76, Y: 11.01, Z: -2.85	0.64	1.80	0.70	Equipment.9	X: 22.76 Y: 11.01, Z: -4.15	0.64	1.80	0.70
C	Start nozzle.1	X: 13.07, Y: 10.26, Z: -4.51				Start nozzle.2	X: 10.32, Y: 10.26, Z: -8.75			
	Start nozzle.3	X: 16.64, Y: 10.51, Z: -13.65				Start nozzle.4	X: 19.50, Y: 10.51, Z: -4.46			
	End nozzle.1	X: 18.99, Y: 11.87, Z: -8.22				End nozzle.2	X: 18.97, Y: 11.87, Z: -11.77			
	End nozzle.3	X: 16.52, Y: 12.40, Z: -6.66				End nozzle.4	X: 12.09, Y: 11.87, Z: -14.97			
	Equipment.1	X: 14.06 Y: 12.10, Z: -10.53	2.42	3.98	3.07	Equipment.2	X: 11.62, Y: 11.12, Z: -14.62	3.61	2.06	1.37
	Equipment.3	X: 19.44, Y: 11.12, Z: -12.12	3.61	2.06	1.37	Equipment.4	X: 19.81, Y: 11.01, Z: -4.40	0.64	1.80	0.70
	Equipment.5	X: 12.76, Y: 11.01, Z: -4.45	0.64	1.80	0.70	Equipment.6	X: 17.07, Y: 11.90, Z: -6.66	1.80	1.25	1.00
	Equipment.7	X: 19.46, Y: 11.12, Z: -8.57	3.61	2.06	1.37	Equipment.8	X: 10.01, Y: 11.01, Z: -8.69	0.64	1.80	0.70
	Equipment.9	X: 16.95, Y: 11.01, Z: -13.59	0.64	1.80	0.70					

Table 5: Performance analysis result of the proposed method.

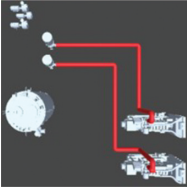
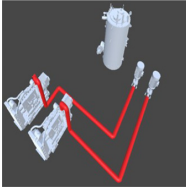
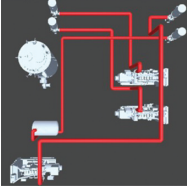
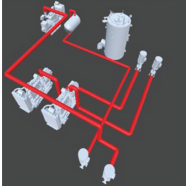
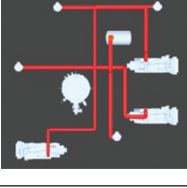
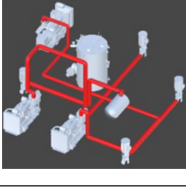
Environment	Top view	Iso view
A		
		
		

Table 6: Objective function values for the proposed method.

Object value/environment	Length (m)	Elbow count (EA)	Path finding time (s)
A	21.66	8	3.8
B	64.24	14	7.3
C	67.12	18	10.2

pensation is high. Moreover, in a large and complex environment, it takes a relatively long time for an agent to determine a path.

Finally, if the progressed learning architecture is drawn, it can be represented as shown in Fig. 15. Here, S,1–10 are the data (distance, index) from the sensor, and D(C, E) is the Manhattan distance between the current position and the position of the end-point. The total number of input data was 21, as described in Section 5.3.1. There are three hidden layers, an input layer, and an output layer. One of 0: up, 1: down, 2: right, 3: left, 4: forward, and 5: backward.

6. Experiments and Results

6.1. Performance analysis of the proposed method

To evaluate the performance of the proposed method, the size of one node was considered as 0.1 m, as shown in Table 4, a $100 \times 100 \times 100$ node environment was constructed, and the experiment was conducted. Table 5 lists the pipe-routing results obtained using the curriculum-based reinforcement-learning method. Here, (A) is the result of finding two paths, (B) is the result of finding the path by adding equipment and routing numbers, and (C) is the result of changing the environment configuration. In addition, Table 6 lists the objective functions of the pipe length, number of elbows, and routing time. Based on the confirmation from Experiment 1, the model learned using the proposed method selected a route at high speed in response to the environment when the routing condition was provided. Even if equipment

is added or the number of paths is changed, the proposed method can quickly respond to the changing environment and select a route.

6.2. Performance analysis of the algorithm

For performance analysis of the algorithms, the environment was configured as shown in Table 7; the size of one node was 0.1 m, and four paths were searched for by each algorithm. Table 8 presents the routing results using A*, JPS, and reinforcement learning based on curriculum learning in the first row, second row, third row, and fourth row, respectively.

Table 9 lists the total length, number of elbows, time taken to select the entire path, and learning status, which are the objective function values for each method. The results indicate that there were slight differences between the methods. There were differences in the number of elbows and the time required to search the path. However, the lengths of all the paths were the same. Those mentioned above and the existing reinforcement-learning method possessed the smallest number of elbows (i.e., 18) followed by the JPS algorithm with 20, and the A* algorithm with 25. Moreover, in terms of speed, the existing reinforcement learning and proposed methods selected the path within 6.7 s, which is the fastest time, followed by JPS (107.68 s) and A* (39 240.72 s). This implies that the proposed method is 16 times faster than the JPS algorithm and 5856 times faster than the A* algorithm in path selection. However, the existing reinforcement-learning-based method and proposed method take time to learn the entire curriculum: 1 h per route for the existing method and 10 h for the proposed method.

The search time of the existing reinforcement-learning method was calculated by adding 1 h of learning time per path. In other words, if the search time per path is 1.7 s, the algorithm takes 3601.7 s. The proposed method required 10 h of training to build a model for path discovery. In other words, if the search time per path was 1.7 s, it took 36 001.7 s. However, in the case of the existing reinforcement-learning method, whenever a path is added, relearning is required. Therefore, 3601.7 s must be added to the path-search time. However, in the case of the proposed methodology, because it is possible to search for a route without re-learning once learning is complete, a time of 1.7 s is added every time a route is added to calculate the route search time. Upon comparing the path-search time considering the aforementioned initial learning time, the existing reinforcement-learning method was found to take 3601.7 s, whereas the proposed method was found to take 36 001.7 s to select a path per one path. In contrast, other methodologies have no learning time; therefore, we checked the search time per path. Because there are countless pipes in a ship, the search speed per number of routes was calculated based on the above results and visualized, as shown in Fig. 16a. The graph was obtained and visualized in log form because the difference in search times was large. In addition, the number of elbows per path was compared, which is the objective function, and visualized as shown in Fig. 16b.

Table 10 is a summary of the data shown in Fig. 16a; the red text indicates cases where other methods were faster than the proposed algorithm, and the yellow text indicates cases where other methods were slower than the proposed algorithm. As illustrated in the results, when the number of routings was small, other methods were on average four times quicker than the A* algorithm and nine times quicker than the existing reinforcement-learning method, which showed better results than the JPS algorithm when the routing count was more than 1427.

Table 7: Environmental conditions for performance analysis by algorithm.

Item.	Point	X-size	Y-size	Z-size	Item.	Point	X-size	Y-size	Z-size
Start nozzle.1	X: 13.14, Y: 10.28, Z: -4.50				Start nozzle.2	X: 13.14, Y: 10.28, Z: -5.75			
End nozzle.1	X: 19.01, Y: 11.94, Z: -7.09				End nozzle.2	X: 19.01, Y: 11.94, Z: -9.46			
Start nozzle.3	X: 22.33, Y: 10.52, Z: -2.95				Start nozzle.4	X: 22.33, Y: 10.52, Z: -4.22			
End nozzle.3	X: 9.97, Y: 11.94, Z: -2.14				End nozzle.4	X: 15.34, Y: 12.32, Z: -8.94			
Equipment.1	X: 10.46, Y: 11.12, Z: -2.47	3.61	2.06	1.37	Equipment.2	X: 15.70, Y: 11.15, Z: -9.24	1.88	2.10	1.81
Equipment.3	X: 12.76, Y: 11.01, Z: -4.45	0.64	1.80	0.70	Equipment.4	X: 19.46, Y: 11.12, Z: -7.45	3.61	2.06	1.37
Equipment.5	X: 12.76, Y: 11.01, Z: -5.75	0.64	1.80	0.70	Equipment.6	X: 19.46, Y: 11.12, Z: -10.11	3.61	2.06	1.37
Equipment.7	X: 11.95, Y: 12.10, Z: -8.93	2.42	3.98	3.07	Equipment.8	X: 22.76, Y: 11.01, Z: -2.85	0.64	1.80	0.70
Equipment.9	X: 15.80, Y: 10.71, Z: -1.69	0.23	1.43	0.23	Equipment.10	X: 22.76, Y: 11.01, Z: -4.15	0.64	1.80	0.70
Equipment.11	X: 15.80 Y: 10.90, Z: -3.14	0.80	1.79	1.50	Equipment.12	X: 18.60 Y: 10.90, Z: -3.14	0.80	1.79	1.50
Equipment.13	X: 15.80 Y: 10.90, Z: -4.64	0.80	1.79	1.50	Equipment.14	X: 18.60 Y: 10.90, Z: -4.64	0.80	1.79	1.50

Table 8: Performance analysis result of the algorithm. RL: Reinforcement learning.

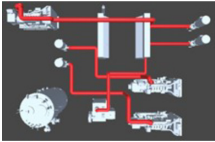
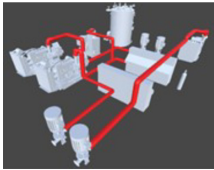
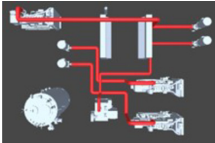
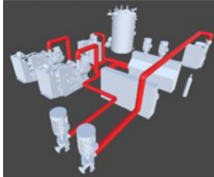
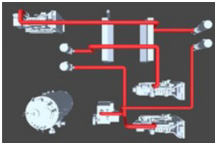
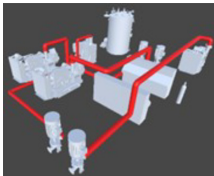
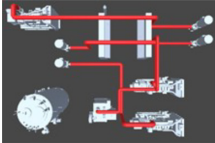
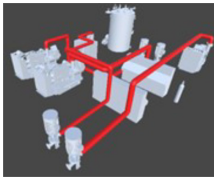
View/algorithm	Top view	Iso view
A*		
JPS		
RL		
RL with curriculum learning		

Table 9: Objective function values for each algorithm.

Object	value/environment Length (m)	Elbow count (EA)	Path finding time (s)	Learning
A*	47.34	25	39 240.72	X
JPS	47.34	20	107.68	X
RL	47.34	18	6.7	O
RL with curriculum learning	47.34	18	6.7	O

The elbow counts in Table 10 summarize the data shown in Fig. 16b. Here, the number of elbows cannot exist as a decimal point. Therefore, we rounded off the obtained value to the nearest integer and compared it. Initially, the A* algorithm had the largest number of bends (seven) and was routable, whereas JPS, reinforcement learning, and the proposed method could find a path with five bends. However, when the number of routings was three or more, the number of elbows for reinforcement learning and the proposed method was reduced compared to other algorithms; therefore, 14 paths could be searched. The difference in elbow count increased gradually as the number of routing operations increased. At routing counts of approximately 2000, the difference between the elbow count of the proposed method and JPS was more than 1000.

7. Conclusions

In this study, we devised a curriculum-based reinforcement-learning method that can quickly respond to environmental changes when selecting a pipeline route for a ship that frequently undergoes modifications. In the proposed method, the avoidance of interference between other routes and existing facilities, minimization of the shape of elbows for cost saving, and minimiza-

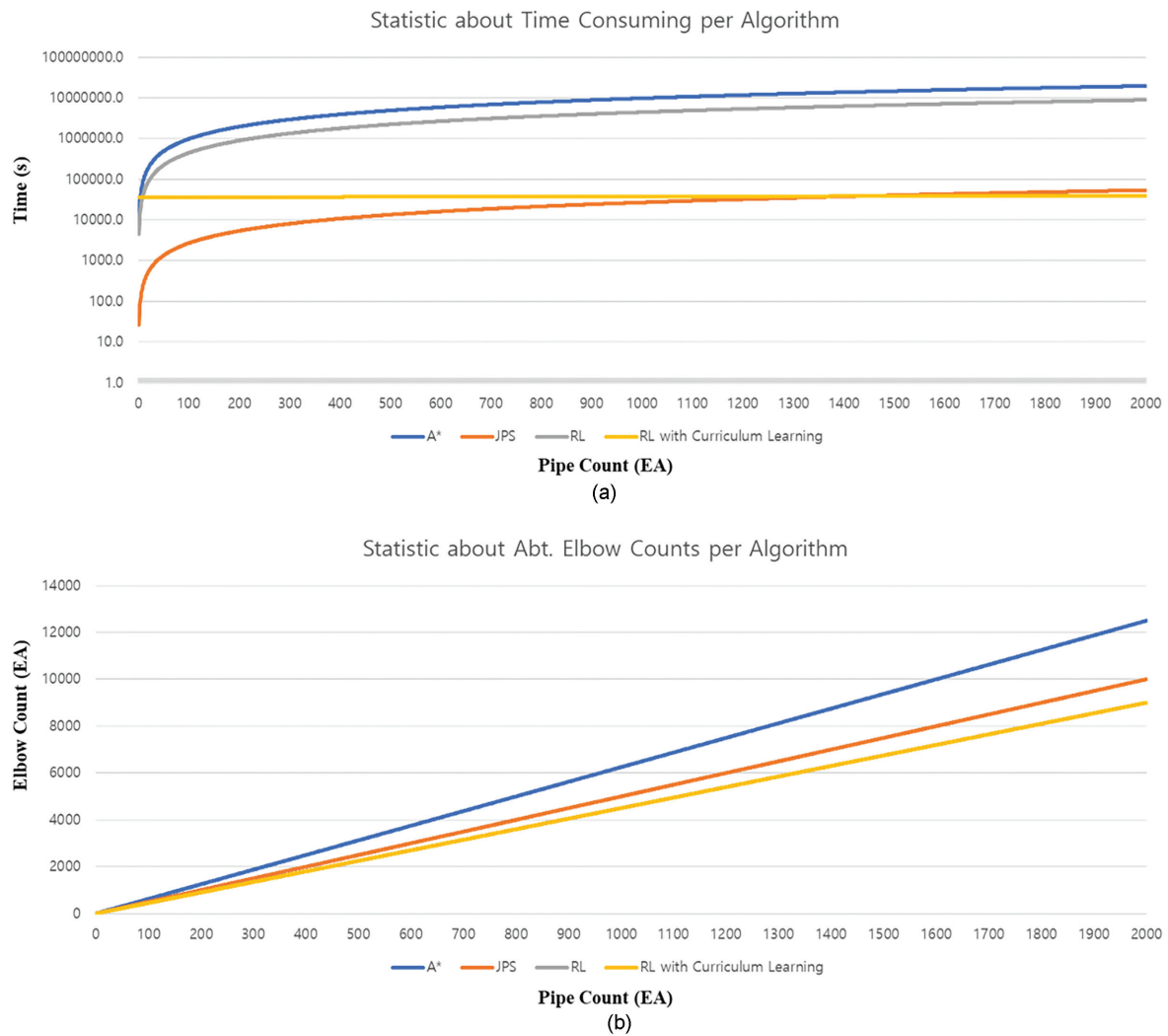


Figure 16: (a) Statistic about time consuming per algorithm, (b) Statistic about Abt. elbow counts per algorithm.

Table 10: Time consumption and elbow count per routing count.

Routing count		A*	JPS		RL		RL with curriculum learning	
Type.	s	Elbow count	s	Elbow count	s	Elbow count	s	Elbow count
1	9810.2	7	26.9	5	3601.7	5	36001.7	5
3	29430.5	19	80.8	15	10805.0	14	36005.0	14
4	39240.7	25	107.7	20	14406.7	18	36006.7	18
8	78481.4	50	215.4	40	28813.4	36	36013.4	36
11	107912.0	57	296.1	45	39618.4	41	36018.4	41
100	981018.0	625	2692.0	500	445667.5	450	36167.5	450
200	1962036.0	1250	5384.0	1000	891335.0	900	36335.0	900
500	4905090.0	3125	13460.0	2500	2228337.5	2250	36837.5	2250
1000	9810180.0	6250	26920.0	5000	4456675.0	4500	37675.0	4500
1426	13989316.7	8913	38387.9	7130	6355218.6	6417	38388.6	6417
1427	13999126.9	8919	38414.8	7135	6359675.2	6422	38390.2	6422
1500	14715270.0	9375	40380.0	7500	6685012.5	6750	38512.5	6750
2000	19620360.0	12500	53840.0	10000	8913350.0	9000	39350.0	9000

tion of the route search time were considered. The first condition (avoidance of interference) was satisfied by setting a constraint such that the learning process was unaffected. The second condition was to minimize the shape of the elbow by assigning a weight

when the shape of the elbow was determined. Finally, in the case of the route search time, methods applying reinforcement learning, which can dynamically search for solutions without calculating the heuristic values of neighboring nodes, can determine a

faster path compared to methods using the existing path-search algorithm.

In addition, to verify practicality of the proposed approach, in Section 6.1, we checked whether route search was possible even when the route was modified. In Section 6.2, routing was performed using the existing methods and the proposed method, and the results were compared and analyzed. Based on the results, it was possible to search for a route with the same length while searching for a route using these methods; however, there was a difference in the bend and search speed. With regard to the number of elbows, the existing reinforcement learning and proposed methods exhibited the smallest elbow and the path was searchable. In addition, in terms of speed, the proposed and existing reinforcement-learning methods performed the best. However, when comparing the route selection time per number of routes, including the learning time, the other methodologies demonstrated excellent results. Nevertheless, when the number of pipes exceeded a specific limit, the proposed method yielded satisfactory search results. Finally, if the proposed algorithm is applied to the pipe-routing task, a piping route can be quickly suggested to the assigned designer, thereby reducing the designer's workload and time required for designing the entire piping route and improving the efficiency of the entire design phase.

Acknowledgments

This work was supported by Inha University Research Grant.

Conflict of interest statement

None declared.

References

- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning* (pp. 41–48). <https://doi.org/10.1145/1553374.1553380>.
- Chun, D.-H., Roh, M.-I., & Lee, H.-W. (2022). Automation of crane control for block lifting based on deep reinforcement learning. *Journal of Computational Design and Engineering*, **9**, 1430–1448. <https://doi.org/10.1093/jcde/qwac063>.
- Fan, X., Lin, Y., & Ji, Z. (2006). The ant colony optimization for ship pipe route design in 3D space. In *Proceedings of the 6th World Congress on Intelligent Control and Automation* (Vol. 1, pp. 3103–3108). <https://doi.org/10.1109/WCICA.2006.1712938>.
- Furuholm, M., Glette, K., Hovin, M., & Torresen, J. (2010). Evolutionary approaches to the three-dimensional multi-pipe routing problem: A comparative study using direct encodings. In *Lecture notes in computer science* (Vol. 6022, pp. 71–82). Springer. https://doi.org/10.1007/978-3-642-12139-5_7.
- Jiang, W. Y., Lin, Y., Chen, M., & Yu, Y. Y. (2015). A co-evolutionary improved multi-ant colony optimization for ship multiple and branch pipe route design. *Ocean Engineering*, **102**, 63–70. <https://doi.org/10.1016/j.oceaneng.2015.04.028>.
- Kim, S.-H. (2014). Development of an automatic pipe routing system on a shipbuilding CAD environment. Ph.D. Thesis, Department of Naval Architecture and Ocean Engineering, Seoul National University. <https://doi.org/10.2478/IJNAOE-2013-0146>.
- Kimura, H. (2011). Automatic designing system for piping and instruments arrangement including branches of pipes. *International Conference on Computer Applications in Shipbuilding*, **3**, 93–99. <https://doi.org/10.2534/JJASNAOE.14.165>.
- Kimura, H., & Ikehira, S. (2009). Automatic design algorithm for pipe arrangement considering valve operability. In *Proceedings of the International Conference on Computer Applications in Shipbuilding* (Vol. 2, pp. 121–126). <https://doi.org/10.2534/jjasnaoe.9.231>.
- Konda, V. R., & Tsitsiklis, J. N. (2003). On Actor-critic algorithms. *Society for Industrial and Applied Mathematics*, **42**, 1143–1166. <https://doi.org/10.1137/S0363012901385691>.
- Lee, J.-B., Roh, M.-I., & Oh, M.-J. (2019). Pipe routing of offshore structure considering space availability. *Korean Journal of Computational Design and Engineering*, **24**, 280–288. <https://doi.org/10.7315/CDE.2019.280>.
- Min, J.-G., Ruy, W.-S., & Park, C. S. (2020). Faster pipe auto-routing using improved jump point search. *International Journal of Naval Architecture and Ocean Engineering*, **12**, 596–604. <https://doi.org/10.1016/j.ijnaoe.2020.07.004>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. *arXiv:1312.5602v1*. <https://doi.org/10.48550/arXiv.1312.5602>.
- Narvekar, S. (2017). Curriculum learning in reinforcement learning. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17)* (pp. 5195–5196). AAAI Press. <https://dl.acm.org/doi/10.5555/3171837.3172051>.
- Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M., & Stone, P. (2020). Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, **21**(181), 1–50. <https://doi.org/10.48550/arXiv.2003.04960>.
- Oh, S. H., Cho, Y. I., & Woo, J. H. (2022). Distributional reinforcement learning with the independent learners for flexible job shop scheduling problem with high variability. *Journal of Computational Design and Engineering*, **9**, 1157–1174. <https://doi.org/10.1093/jcde/qwac044>.
- Park, J.H., & Storch, R. (2002). Pipe-routing algorithm development: Case study of a ship engine room design. *Journal of the Expert Systems with Applications*, **23**, 299–309. [https://doi.org/10.1016/S0957-4174\(02\)00049-0](https://doi.org/10.1016/S0957-4174(02)00049-0).
- Schulman, J., Levine, S., Moritz, P., Jordan, M., & Abbeel, P. (2017). Trust region policy optimization. *arXiv:1502.05477v5*. <https://doi.org/10.48550/arXiv.1502.05477>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347v2*. <https://doi.org/10.48550/arXiv.1707.06347>.
- Shin, D.-s., Park, B.-c., Lim, C., Oh, S.-j., Kim, G.-y., & Shin, S.-c. (2020). Pipe routing using reinforcement learning on initial design stage. *Journal of the Society of Naval Architects of Korea*, **57**, 191–197. <https://doi.org/10.3744/SSNAK.2020.57.4.191>.
- Sui, H., & Niu, W. (2016). Branch-pipe-routing approach for ships using improved genetic algorithm. *Frontiers of Mechanical Engineering*, **11**, 316–323. <https://doi.org/10.1007/s11465-016-0384-z>.